

编译原理

第八讲：程序运行时的存储组织及管理

授课教师：柯嵩宇

福州大学计算机与大数据学院

2026 年 4 月 30 日



福州大学
FUZHOU UNIVERSITY

目录

- ① 运行时的存储组织的作用与任务
- ② 存储分配策略
- ③ 活动记录
- ④ 函数调用与参数传递

- ⑤ 垃圾回收
- ⑥ OOP 的运行时组织
- ⑦ 小结与作业



目录

① 运行时的存储组织的作用与任务

② 存储分配策略

③ 活动记录

④ 函数调用与参数传递

⑤ 垃圾回收

⑥ OOP 的运行时组织

⑦ 小结与作业



运行时存储管理的作用与任务

核心作用

连接编译时语用分析与目标代码执行的桥梁，负责将源代码中的名字 (**Name**) 映射到运行时的地址 (**Address**)。

运行时存储管理的作用与任务

核心作用

连接编译时语用分析与目标代码执行的桥梁，负责将源代码中的名字 (**Name**) 映射到运行时的地址 (**Address**)。

具体任务

- 空间分配：为源程序中的数据对象分配内存空间。

运行时存储管理的作用与任务

核心作用

连接编译时语用分析与目标代码执行的桥梁，负责将源代码中的名字 (**Name**) 映射到运行时的地址 (**Address**)。

具体任务

- 空间分配：为源程序中的数据对象分配内存空间。
- 名字绑定：管理变量名与其存储地址的绑定关系。

运行时存储管理的作用与任务

核心作用

连接编译时语用分析与目标代码执行的桥梁，负责将源代码中的名字 (**Name**) 映射到运行时的地址 (**Address**)。

具体任务

- 空间分配：为源程序中的数据对象分配内存空间。
- 名字绑定：管理变量名与其存储地址的绑定关系。
- 过程支持：支持函数调用、参数传递、递归调用。

运行时存储管理的作用与任务

核心作用

连接编译时语用分析与目标代码执行的桥梁，负责将源代码中的名字 (**Name**) 映射到运行时的地址 (**Address**)。

具体任务

- 空间分配：为源程序中的数据对象分配内存空间。
- 名字绑定：管理变量名与其存储地址的绑定关系。
- 过程支持：支持函数调用、参数传递、递归调用。
- 生命周期管理：控制对象在内存中的创建与撤销。

目录

① 运行时的存储组织的作用与任务

② 存储分配策略

安全机制：地址空间布局随机化 (ASLR)

安全机制：不可执行内存 ($W \oplus X$)

③ 活动记录

④ 函数调用与参数传递

⑤ 垃圾回收

⑥ OOP 的运行时组织

⑦ 小结与作业



内存空间布局

低地址

代码区 (Code Segment) - 只读

静态数据区 (Static Data)

堆 (Heap) ↓

自由空间

高地址

栈 (Stack) ↑

三种主要的存储分配策略

策略	特点	适用场景
静态存储分配	编译时确定地址，运行期固定不变	全局变量、静态变量
栈式存储分配	随函数调用进栈/出栈，支持递归	局部变量、实参、返回地址
堆式存储分配	运行时按需申请，需垃圾回收支持	动态数据结构、OOP 对象

栈式与堆式存储分配对比

栈式分配 (Stack)

- LIFO (后进先出) 结构。
- 效率极高，地址连续。
- **无法支持**：生命周期长于过程调用的数据。

栈式与堆式存储分配对比

栈式分配 (Stack)

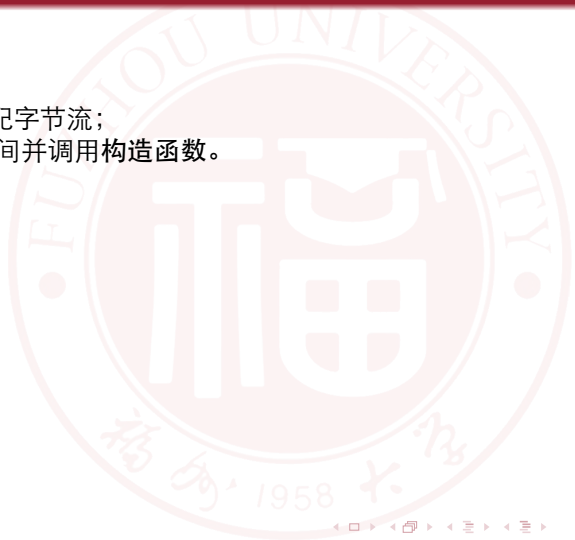
- LIFO (后进先出) 结构。
- 效率极高，地址连续。
- **无法支持**：生命周期长于过程调用的数据。

堆式分配 (Heap)

- 任意顺序分配和释放。
- 需管理内存碎片。
- **核心技术**：垃圾回收 (GC)。

C++ 内存分配：new vs malloc

- 本质区别：
 - malloc 是标准库函数，仅分配字节流；
 - new 是 C++ 运算符，分配空间并调用构造函数。



C++ 内存分配: new vs malloc

- 本质区别:
 - malloc 是标准库函数, 仅分配字节流;
 - new 是 C++ 运算符, 分配空间并调用构造函数。
- 分配分水岭 (以 glibc 为例):

申请大小	系统调用	内存区域
小块 (< 128 KB)	brk()	堆 (Heap)
大块 (> 128 KB)	mmap()	映射区 (Mmap Segment)

C++ 内存分配: new vs malloc

- 本质区别:
 - malloc 是标准库函数, 仅分配字节流;
 - new 是 C++ 运算符, 分配空间并调用构造函数。
- 分配分水岭 (以 glibc 为例):

申请大小	系统调用	内存区域
小块 (< 128 KB)	brk()	堆 (Heap)
大块 (> 128 KB)	mmap()	映射区 (Mmap Segment)

- 管理策略:
 - brk 通过推高堆顶指针分配, 释放后内存常留在池中以减少碎片;
 - mmap 直接请求独立虚拟地址, 释放后立即归还内核。

Linux mmap 机制深度解析

核心理念：虚拟地址映射

mmap 将一段虚拟内存映射到具体目标（文件或匿名空间），并不立即分配物理内存。

- 两种映射形态：

Linux mmap 机制深度解析

核心理念：虚拟地址映射

mmap 将一段虚拟内存映射到具体目标（文件或匿名空间），并不立即分配物理内存。

- 两种映射形态：

- ① 文件映射：磁盘文件直接映射到内存，实现零拷贝 (Zero-copy) 访问。

Linux mmap 机制深度解析

核心理念：虚拟地址映射

mmap 将一段虚拟内存映射到具体目标（文件或匿名空间），并不立即分配物理内存。

- 两种映射形态：

- ① 文件映射：磁盘文件直接映射到内存，实现零拷贝 (Zero-copy) 访问。
- ② 匿名映射：不关联文件，用于 malloc 申请大块内存。

Linux mmap 机制深度解析

核心理念：虚拟地址映射

mmap 将一段虚拟内存映射到具体目标（文件或匿名空间），并不立即分配物理内存。

- 两种映射形态：
 - ① 文件映射：磁盘文件直接映射到内存，实现零拷贝 (Zero-copy) 访问。
 - ② 匿名映射：不关联文件，用于 malloc 申请大块内存。
- 缺页中断 (Page Fault) 流程：

Linux mmap 机制深度解析

核心理念：虚拟地址映射

mmap 将一段虚拟内存映射到具体目标（文件或匿名空间），并不立即分配物理内存。

- 两种映射形态：
 - ① 文件映射：磁盘文件直接映射到内存，实现零拷贝 (Zero-copy) 访问。
 - ② 匿名映射：不关联文件，用于 malloc 申请大块内存。
- 缺页中断 (Page Fault) 流程：
 - 懒加载：申请时仅在页表中“挂名”。

Linux mmap 机制深度解析

核心理念：虚拟地址映射

mmap 将一段虚拟内存映射到具体目标（文件或匿名空间），并不立即分配物理内存。

- 两种映射形态：
 - ① 文件映射：磁盘文件直接映射到内存，实现零拷贝 (Zero-copy) 访问。
 - ② 匿名映射：不关联文件，用于 malloc 申请大块内存。
- 缺页中断 (Page Fault) 流程：
 - 懒加载：申请时仅在页表中“挂名”。
 - 实地分配：只有当进程真正访问该地址时，触发缺页中断。

Linux mmap 机制深度解析

核心理念：虚拟地址映射

mmap 将一段虚拟内存映射到具体目标（文件或匿名空间），并不立即分配物理内存。

- 两种映射形态：
 - ① 文件映射：磁盘文件直接映射到内存，实现零拷贝 (Zero-copy) 访问。
 - ② 匿名映射：不关联文件，用于 malloc 申请大块内存。
- 缺页中断 (Page Fault) 流程：
 - 懒加载：申请时仅在页表中“挂名”。
 - 实地分配：只有当进程真正访问该地址时，触发缺页中断。
 - 内核介入：内核捕获异常，分配物理页，更新页表，恢复运行。

ptmalloc 算法与现代分配器

- **ptmalloc 核心算法 (Bins 机制):**
 - **Fast Bins:** LIFO 队列, 处理极小内存, 不合并。
 - **Unsorted Bin:** 作为最近释放块的缓冲区, 提升复用速度。
 - **Small/Large Bins:** 分级存储, 采用 FIFO 或大小排序进行匹配。
- 常见分配算法对比:

算法/分配器	核心特点	应用场景
Buddy System	物理页面二分合并	Linux 内核底层
TCMalloc	每个线程私有缓存, 无锁分配	高并发服务器 (Google)
JEMalloc	Arena 分级, 防碎片能力强	FreeBSD, Rust, Redis

- 编译器视角: 若实现自动回收, 需考虑 标记-清除 或 分代回收 (GC) 算法。

目录

① 运行时的存储组织的作用与任务

② 存储分配策略

安全机制：地址空间布局随机化 (ASLR)

安全机制：不可执行内存 ($W \oplus X$)

③ 活动记录

④ 函数调用与参数传递

⑤ 垃圾回收

⑥ OOP 的运行时组织

⑦ 小结与作业

内存安全：ASLR 的引入

传统固定布局 (Fixed Layout)

- 栈、堆、代码段地址在每次运行时完全相同。
- **风险**：攻击者通过溢出漏洞，可以精确预测返回地址或跳转到特定的 Libc 函数（如 `system`）。

ASLR 布局 (Randomized)

- 操作系统在每次程序启动时，为各内存段设置随机的基地址（Offset）。
- **优势**：攻击者无法预知目标地址，极大地提升了 Return-to-libc 等攻击的门槛。



ASLR 与 PIE 的协作

- **ASLR (内核支持)**: 负责将栈、堆、动态链接库地址随机化。
- **PIE (位置无关可执行程序)**:
 - 编译器生成的代码不依赖绝对地址。
 - 使得可执行文件本身的代码段也可以在内存中随机摆放。

ASLR 与 PIE 的协作

- **ASLR (内核支持)**: 负责将栈、堆、动态链接库地址随机化。
- **PIE (位置无关可执行程序)**:
 - 编译器生成的代码不依赖绝对地址。
 - 使得可执行文件本身的代码段也可以在内存中随机摆放。

小提示

即使有了 ASLR，如果程序没有开启 PIE，那么代码段（Text Segment）的地址依然是固定的，攻击者可以利用代码段中的 Gadgets 进行 ROP 攻击。

目录

① 运行时的存储组织的作用与任务

② 存储分配策略

安全机制：地址空间布局随机化 (ASLR)

安全机制：不可执行内存 ($W \oplus X$)

③ 活动记录

④ 函数调用与参数传递

⑤ 垃圾回收

⑥ OOP 的运行时组织

⑦ 小结与作业

内存安全：W $\oplus$ X (NX/DEP) 机制

核心原则：互斥性

在任意给定的时刻，一个内存页面要么是可写 (**Writable**) 的，要么是可执行 (**Executable**) 的，但不能同时两者皆是。

$$\text{Writable} \oplus \text{Executable} = \text{True}$$

内存安全：W $\oplus$ X (NX/DEP) 机制

核心原则：互斥性

在任意给定的时刻，一个内存页面要么是可写 (**Writable**) 的，要么是可执行 (**Executable**) 的，但不能同时两者皆是。

$$\text{Writable} \oplus \text{Executable} = \text{True}$$

- 传统风险：攻击者将恶意代码 (Shellcode) 写入栈中，然后通过溢出修改返回地址，跳转到栈上执行。

内存安全: $W \oplus X$ (NX/DEP) 机制

核心原则: 互斥性

在任意给定的时刻, 一个内存页面要么是可写 (**Writable**) 的, 要么是可执行 (**Executable**) 的, 但不能同时两者皆是。

$$\text{Writable} \oplus \text{Executable} = \text{True}$$

- 传统风险: 攻击者将恶意代码 (Shellcode) 写入栈中, 然后通过溢出修改返回地址, 跳转到栈上执行。
- 防护原理:
 - 栈和堆被标记为 **RW** (不可执行)。
 - 代码段被标记为 **RX** (不可写)。

内存安全: $W \oplus X$ (NX/DEP) 机制

核心原则: 互斥性

在任意给定的时刻, 一个内存页面要么是可写 (**Writable**) 的, 要么是可执行 (**Executable**) 的, 但不能同时两者皆是。

$$\text{Writable} \oplus \text{Executable} = \text{True}$$

- 传统风险: 攻击者将恶意代码 (Shellcode) 写入栈中, 然后通过溢出修改返回地址, 跳转到栈上执行。
- 防护原理:
 - 栈和堆被标记为 **RW** (不可执行)。
 - 代码段被标记为 **RX** (不可写)。
- 硬件支持: Intel 称为 **NX (No-Execute)** 位, AMD 称为 **EVP**, Windows 称为 **DEP** (数据执行保护)。

W ⊕ X 的局限性与防御演进

- 被动进化：既然攻击者不能在栈上执行代码，他们便转向了 **Code Reuse Attack** (代码复用攻击)。
- 典型手段：**ROP (Return-Oriented Programming)**。
 - 攻击者利用代码段中已有的合法指令片段 (Gadgets)，通过精心构造的返回地址序列拼凑出恶意功能。

W ⊕ X 的局限性与防御演进

- 被动进化：既然攻击者不能在栈上执行代码，他们便转向了 **Code Reuse Attack** (代码复用攻击)。
- 典型手段：**ROP (Return-Oriented Programming)**。
 - 攻击者利用代码段中已有的合法指令片段 (Gadgets)，通过精心构造的返回地址序列拼凑出恶意功能。

编译器与安全的平衡

现代编译器（如 GCC/Clang）通过 `-z noexecstack` 标志默认启用此特性。但在某些场景下（如 **JIT** 即时编译，Java/JavaScript V8 引擎），需要在运行时动态生成代码，必须小心地在 W 和 X 权限之间切换。

目录

① 运行时的存储组织的作用与任务

② 存储分配策略

③ 活动记录

活动记录的创建

活动记录的撤销与删除

活动记录中的局部数据与非局部量
访问

④ 函数调用与参数传递

⑤ 垃圾回收

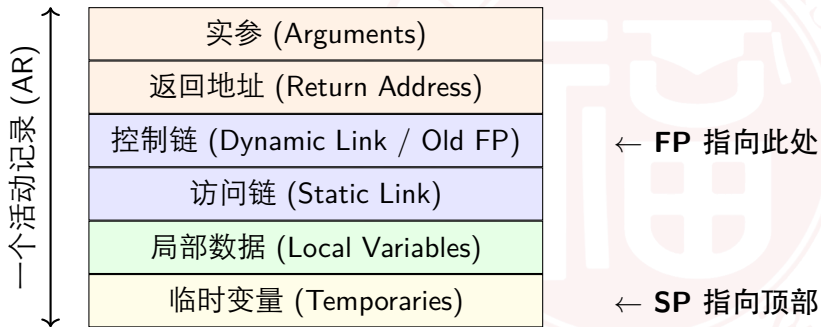
⑥ OOP 的运行时组织

⑦ 小结与作业



活动记录的完整结构

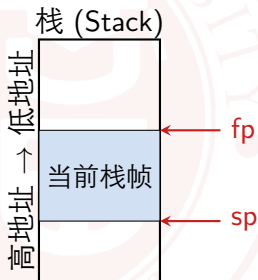
一个完整的活动记录在栈上的布局通常如下：



核心指针：fp 与 sp

在栈式存储管理中，我们依赖两个关键寄存器来定位活动记录：

- **sp (Stack Pointer):**
 - 始终指向当前栈的顶部（最新分配单元）。
 - 随数据的压入 (Push) 和弹出 (Pop) 动态移动。
- **fp (Frame Pointer):**
 - 指向当前活动记录的基地址（固定位置）。
 - 用于通过“基址 + 偏移量”访问局部变量和参数。
 - 在子程序执行期间保持不变。



目录

① 运行时的存储组织的作用与任务

② 存储分配策略

③ 活动记录

活动记录的创建

活动记录的撤销与删除

活动记录中的局部数据与非局部量
访问

④ 函数调用与参数传递

⑤ 垃圾回收

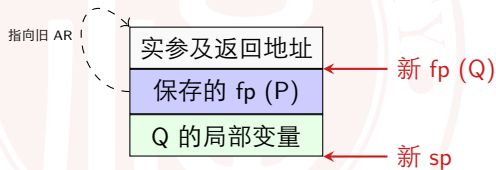
⑥ OOP 的运行时组织

⑦ 小结与作业

创建栈帧：被调用者的交接

当控制流进入 Q 后，执行“序言 (Prologue)”代码：

- ① 保存旧 **fp**：将 P 的 fp 压栈（即控制链）。
- ② 更新 **fp**：令 $fp = sp$ ，标记新栈帧的开始。
- ③ 分配空间：移动 sp 为局部变量预留空间。



目录

① 运行时的存储组织的作用与任务

② 存储分配策略

③ 活动记录

活动记录的创建

活动记录的撤销与删除

活动记录中的局部数据与非局部量
访问

④ 函数调用与参数传递

⑤ 垃圾回收

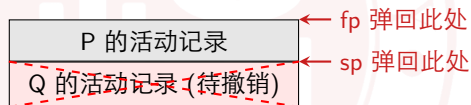
⑥ OOP 的运行时组织

⑦ 小结与作业

撤销栈帧：函数返回

当 Q 执行结束，通过“结语 (Epilogue)”代码清理现场：

- ① 恢复 sp ：令 $sp = fp$ 。此时局部变量空间被逻辑撤销。
- ② 恢复旧 fp ：从栈中弹出旧值并赋给 fp 寄存器。
- ③ 跳转返回：弹出返回地址，恢复 P 的执行。



目录

① 运行时的存储组织的作用与任务

② 存储分配策略

③ 活动记录

活动记录的创建

活动记录的撤销与删除

活动记录中的局部数据与非局部量
访问

④ 函数调用与参数传递

⑤ 垃圾回收

⑥ OOP 的运行时组织

⑦ 小结与作业

活动记录中的局部数据示例

考虑如下 C 语言函数：

```
1  int foo(int a, int b) {  
2      int x = a + b;  
3      int y = x * 2;  
4      return y;  
5  }
```

参数 a, b
返回地址
旧 fp
局部变量 x, y
临时空间

- 局部变量 x, y 在函数进入时分配。
- 通过 **fp** - 偏移量 访问（具体偏移由编译器计算）。
- 函数返回时，整块空间随栈帧一起撤销。

嵌套过程定义中的非局部量访问

问题背景

Pascal 等语言允许过程嵌套定义。内层过程可能需要访问外层过程的局部变量。

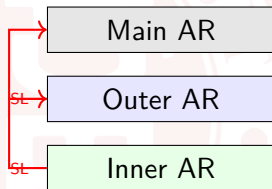
示例

```
1 procedure Outer;  
2   var x: integer;  
3   procedure Inner;  
4     begin x := x + 1 end;  
5 begin ... Inner; ... end;
```

- Inner 需要访问 Outer 的局部变量 x。
- 仅通过控制链（Control Link）只能找到调用者，不能找到定义外层。

静态链 (Static Link / Access Link)

- 静态链指向词法上直接外层过程的活动记录。
- 用于在静态作用域下查找非局部量。
- 访问深度为 k 的外层变量，需沿静态链跳转 k 次。



Display 表优化

静态链的缺陷：访问非局部量需要沿链多次跳转，时间复杂度为 $O(n)$ 。

Display 表

用一个数组 $D[]$ 记录每个嵌套层当前最新活动记录的地址。

- $D[i]$ 指向嵌套深度为 i 的过程的当前 AR。
- 访问深度为 k 的非局部量变为 $O(1)$ ：直接通过 $D[k]$ 定位。

Display 表优化

静态链的缺陷：访问非局部量需要沿链多次跳转，时间复杂度为 $O(n)$ 。

Display 表

用一个数组 $D[]$ 记录每个嵌套层当前最新活动记录的地址。

- $D[i]$ 指向嵌套深度为 i 的过程的当前 AR。
- 访问深度为 k 的非局部量变为 $O(1)$ ：直接通过 $D[k]$ 定位。

维护开销

每次过程调用/返回时需更新 Display 表，但现代编译器通常认为这种开销是值得的。

嵌套语句块的非局部量访问

某些语言（如 C、Java）允许在函数内部嵌套语句块（Block）：

```
1 void foo() {  
2     int a = 10;  
3     {      // 嵌套块开始  
4         int b = 20;  
5         a = a + b;  // 访问外层变量 a  
6     }      // 嵌套块结束  
7 }
```

- 嵌套块通常不创建新的活动记录，而是复用外层函数的 AR。

嵌套语句块的非局部量访问

某些语言（如 C、Java）允许在函数内部嵌套语句块（Block）：

```
1 void foo() {  
2     int a = 10;  
3     {      // 嵌套块开始  
4         int b = 20;  
5         a = a + b;  // 访问外层变量 a  
6     }      // 嵌套块结束  
7 }
```

- 嵌套块通常不创建新的活动记录，而是复用外层函数的 AR。
- 块的局部变量在 AR 中分配，块结束时逻辑上“销毁”（通常仅调整 SP，不实际清零）。

嵌套语句块的非局部量访问

某些语言（如 C、Java）允许在函数内部嵌套语句块（Block）：

```
1 void foo() {  
2     int a = 10;  
3     { // 嵌套块开始  
4         int b = 20;  
5         a = a + b; // 访问外层变量 a  
6     } // 嵌套块结束  
7 }
```

- 嵌套块通常不创建新的活动记录，而是复用外层函数的 AR。
- 块的局部变量在 AR 中分配，块结束时逻辑上“销毁”（通常仅调整 SP，不实际清零）。
- 非局部量访问通过同一 AR 内的偏移量直接定位。

动态作用域规则下的非局部量访问

动态作用域 (Dynamic Scoping)

变量的绑定取决于运行时调用链，而非词法嵌套结构。

查找规则：

- ① 先在当前 AR 中查找。
- ② 若未找到，沿控制链 (**Control Link**) 向上查找调用者的 AR。
- ③ 直到找到或到达全局作用域。

代表语言：

- 早期 Lisp 方言
- Emacs Lisp
- 部分 Shell 脚本

对比

- 静态作用域：编译时可确定偏移，效率高。
- 动态作用域：运行时才能确定绑定，调试困难但灵活性高。

总结：栈帧的生命周期

阶段	寄存器动作	内存效果
调用发生	sp 向低地址移动	压入参数与返回地址
序言代码	save fp; fp = sp	建立新帧基址
函数执行	fp 不变, sp 随临时运算波动	稳定访问局部量
结语代码	sp = fp; pop fp	销毁当前帧, 恢复调用者环境

关键点

只要 sp 移动到活动记录基地址上方, 该记录就在逻辑上被“撤销”了, 空间可被后续调用重用。

目录

① 运行时的存储组织的作用与任务

② 存储分配策略

③ 活动记录

④ 函数调用与参数传递

x86 调用约定详解

RISC 架构下的函数调用：ARM,
MIPS, RISC-V

⑤ 垃圾回收

⑥ OOP 的运行时组织

⑦ 小结与作业



典型的函数调用序列

调用者 (Caller) 任务

- ① 计算实参并压栈。
- ② 保存返回地址及当前寄存器。
- ③ 跳转至子程序。

被调用者 (Callee) 任务

- ① 保存旧 AR 指针 (Control Link)。
- ② 初始化局部变量。
- ③ 执行函数体。
- ④ 将结果放入预定寄存器/位置。

参数传递方式

- 传值 (Call by Value): 传递副本, 函数内修改不影响原件。
- 传地址/引用 (Call by Reference): 传递变量地址, 直接操作原件。
- 复写-回写 (Copy-in/Copy-out): 进入时拷贝, 退出时写回原变量。
- 传名 (Call by Name): Algol 60 特性, 类似于宏替换, 每次访问形参时重新计算实参表达式。

目录

① 运行时的存储组织的作用与任务

② 存储分配策略

③ 活动记录

④ 函数调用与参数传递

x86 调用约定详解

RISC 架构下的函数调用：ARM,
MIPS, RISC-V

⑤ 垃圾回收

⑥ OOP 的运行时组织

⑦ 小结与作业



为什么需要调用约定 (Calling Convention)

- 协议定义：调用者与被调用者之间关于“如何传递参数”和“如何清理环境”的契约。
- 核心解决问题：
 - 参数压栈的顺序（从左到右还是从右到左?）。
 - 参数是通过栈传递还是寄存器传递？
 - 栈平衡责任：由 Caller 还是 Callee 负责将 SP 恢复原状？
 - 返回值的传递方式（通常是 EAX）。

常见调用约定对比: cdecl vs. stdcall

cdecl (C Declaration)

- 来源: C 语言默认约定。
- 参数顺序: 从右向左压栈。
- 栈平衡: **Caller (调用者)** 负责。
- 优点: 支持变长参数 (如 `printf`), 因为只有调用者知道压了多少参数。

stdcall (Standard Call)

- 来源: Windows API 常用。
- 参数顺序: 从右向左压栈。
- 栈平衡: **Callee (被调用者)** 负责。
- 优点: 生成的代码体积稍小 (清理代码只在 Callee 出现一次)。

高效率约定：fastcall

- 核心思想：内存访问慢，寄存器访问快。
- 策略：
 - 前两个（或若干个）参数通过 **ECX** 和 **EDX** 寄存器传递。
 - 剩余参数依然通过栈传递（从右向左）。
- 栈平衡：被调用者清理栈。

现代编译器趋势

在 x64 体系下，调用约定得到了统一（如 Windows x64 或 System V AMD64 ABI），大量依赖寄存器传递，进一步淡化了 cdecl 和 stdcall 的区别。

栈平衡的底层指令差异

cdecl (Caller 清理):

```
1 push arg2
2 push arg1
3 call function
4 add esp, 8 ; 调用者清理栈
```

stdcall (Callee 清理):

```
1 push arg2
2 push arg1
3 call function
4 ; 栈已在函数内部平衡
5
6 ; --- 函数内部结语 ---
7 ret 8 ; 弹出返回地址后
8 ; 将 esp 增加 8
```

参数传递方式总结表

约定	参数压栈顺序	清理者	寄存器使用	典型应用
__cdecl	自右向左	调用者	无	C/C++ 默认
__stdcall	自右向左	被调用者	无	Win32 API
__fastcall	自右向左	被调用者	ECX, EDX	高性能函数
__thiscall	自右向左	被调用者	ECX (this)	C++ 非静态成员函数

目录

① 运行时的存储组织的作用与任务

② 存储分配策略

③ 活动记录

④ 函数调用与参数传递

x86 调用约定详解

RISC 架构下的函数调用: ARM,
MIPS, RISC-V

⑤ 垃圾回收

⑥ OOP 的运行时组织

⑦ 小结与作业

RISC 与 x86 函数调用的根本差异

x86 (CISC) 风格

- 频繁访问内存（压栈/出栈）。
- `call` 指令自动将返回地址压栈。
- 参数主要通过栈传递。

RISC 风格 (ARM/MIPS/RISC-V)

- 寄存器传参：前 4 或 8 个参数直接进寄存器。
- 链接寄存器 (**LR**)：返回地址先存入寄存器，不进内存。
- 叶函数 (**Leaf**) 优化：不调用其他函数时，甚至不需要分配栈空间。

ARM (AArch64) 函数调用约定

- 参数寄存器：使用 X0-X7 传递前 8 个参数。超过部分使用栈。
- 返回地址：保存在 X30 (LR) 寄存器中。
- 栈平衡：栈指针 SP 必须保持 16 字节对齐。

过程调用指令

使用 BL (Branch with Link) 跳转，该指令自动将下一条指令地址存入 X30。

MIPS 函数调用与“延迟槽”

- 参数寄存器：使用 \$a0-\$a3 传递前 4 个参数。
- 返回值：保存在 \$v0-\$v1。
- 返回地址：保存在 \$ra。

寄存器编号	助记符	用途
\$2 - \$3	\$v0-\$v1	函数返回值
\$4 - \$7	\$a0-\$a3	函数参数
\$31	\$ra	返回地址 (Return Address)

RISC-V 调用约定 (RV64I)

RISC-V 采用了非常现代且整齐的寄存器规划：

- 参数/返回值：a0-a7 (共 8 个寄存器)。其中 a0, a1 同时也作为返回值寄存器。
- 临时寄存器：t0-t6 (Caller-saved) 和 s0-s11 (Callee-saved)。
- 栈指针与帧指针：sp (x2), fp (x8, 即 s0)。

ABI Name	Role
ra	Return Address
sp	Stack Pointer
gp/tp	Global/Thread Ptr
a0-a7	Args/Results

深度解析：叶函数 (Leaf Function) 的特殊性

什么是叶函数？不调用任何其他过程的过程。

RISC 的优化策略

- 无需保存 **LR**：返回地址就在寄存器里，直接 `ret` 或 `jr ra`。
- 无需压栈：如果寄存器足够用，该函数完全不在内存中分配栈帧。
- 性能提升：减少了内存访问，降低了缓存失效。

Listing 1: 叶函数示例

```
1  # int add(int a, int b)
2  add:
3      add a0, a0, a1
4      ret    # 直接跳转 ra
```

三大架构调用约定对比总结

特性	ARM (A64)	MIPS	RISC-V
参数寄存器	X0 - X7 (8 个)	\$a0 - \$a3 (4 个)	a0 - a7 (8 个)
返回地址寄存器	X30 (LR)	\$31 (\$ra)	x1 (ra)
帧指针寄存器	X29 (FP)	\$30 (\$s8)	x8 (fp/s0)
跳转指令	BL (Branch with Link)	JAL (Jump and Link)	JALR / CALL
栈对齐要求	16 字节	8 字节	16 字节

目录

- ① 运行时的存储组织的作用与任务
- ② 存储分配策略
- ③ 活动记录
- ④ 函数调用与参数传递

⑤ 垃圾回收

⑥ OOP 的运行时组织

⑦ 小结与作业



为什么需要垃圾回收

手动内存管理的困境

- 内存泄漏：分配后忘记释放，导致可用内存持续减少。
- 悬空指针：释放后仍被引用，访问已回收空间。
- 重复释放：同一内存被释放两次，破坏堆结构。

为什么需要垃圾回收

手动内存管理的困境

- 内存泄漏：分配后忘记释放，导致可用内存持续减少。
- 悬空指针：释放后仍被引用，访问已回收空间。
- 重复释放：同一内存被释放两次，破坏堆结构。

垃圾回收 (Garbage Collection, GC) 的核心思想

由运行时系统自动识别并回收不再被程序使用的内存对象，解放程序员的心智负担。

垃圾的定义与判定标准

定义

垃圾 (Garbage): 程序中再也不可能被访问到的内存对象。

可达性分析 (Reachability Analysis):

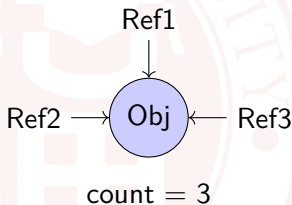
- **根集合 (Root Set):** 全局变量、寄存器、栈上的局部变量等直接可达的对象。
- **可达对象:** 从根集合出发, 通过引用链能够到达的对象。
- **垃圾对象:** 不可达的对象即为垃圾。

引用计数 (Reference Counting)

基本原理

为每个对象维护一个引用计数器，记录指向该对象的引用数量。

- 新引用指向对象时：计数 +1
- 引用失效或被覆盖时：计数 -1
- 计数归零时：立即回收该对象



引用计数的优缺点

优点

- 垃圾回收及时，对象变垃圾后立即被回收。
- 回收操作分散在运行时，无全局停顿。
- 实现简单，适合实时性要求高的场景。

缺点

- 循环引用无法解决：A 引用 B，B 引用 A，但两者均不可达。
- 每次引用修改都有额外开销。
- 需要额外空间存储计数器。

标记-清除 (Mark-Sweep)

核心思想

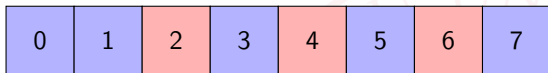
分两个阶段处理：

- ① 标记阶段 (Mark)：从根集合出发，遍历所有可达对象并标记。
- ② 清除阶段 (Sweep)：遍历整个堆，回收未标记的对象。

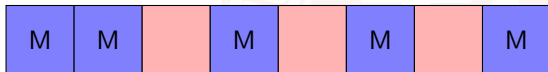
主要问题：内存碎片

回收后产生大量不连续的内存碎片，可能导致大对象无法分配，即使总空闲内存足够。

标记-清除过程示意图



初始状态（蓝色=存活，红色=垃圾）



标记阶段



清除阶段（白色=空闲碎片）

复制回收 (Copying Collection)

核心思想

将堆划分为两个相等的半区 (Semi-space): **From-space** 和 **To-space**。

- 只在 From-space 中分配对象。
- GC 时, 将存活对象复制到 To-space, 然后交换两个半区。

优点

- 无内存碎片。
- 分配速度快 (指针碰撞)。

缺点

- 内存利用率仅 50%。
- 存活对象多时复制开销大。

分代回收 (Generational GC)

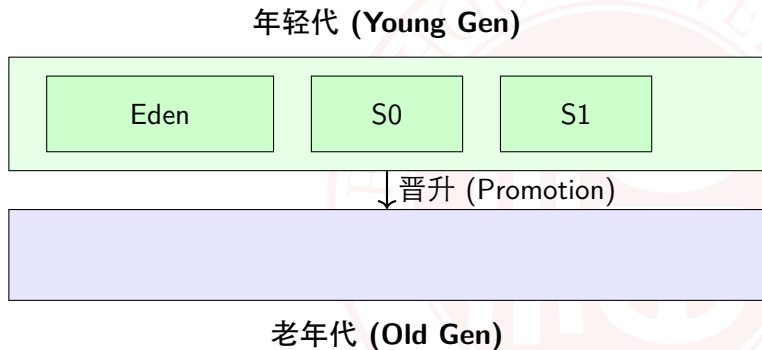
代际假说 (Generational Hypothesis)

- 大多数对象生命周期很短。
- 越老的对象越不容易死去。

堆的划分：

- 年轻代 (**Young Generation**)：存放新分配的对象，GC 频率高，使用复制算法。
- 老年代 (**Old Generation**)：存放经过多次 GC 仍存活的对象，GC 频率低，使用标记-清除或标记-整理。

分代回收示意图



对象在 Eden 分配，经过 Minor GC 存活后进入 Survivor 区，多次存活后晋升到老年代。

增量回收 (Incremental GC)

问题背景

传统 GC（如标记-清除）需要暂停应用线程 (Stop-The-World)，导致明显的卡顿。

增量回收的核心思想

将 GC 工作拆分成多个小步骤，与应用线程交替执行，减少单次停顿时间。

- 三色标记法：白色（未访问）、灰色（已访问，子对象未完全处理）、黑色（已完成）。

增量回收 (Incremental GC)

问题背景

传统 GC（如标记-清除）需要暂停应用线程 (Stop-The-World)，导致明显的卡顿。

增量回收的核心思想

将 GC 工作拆分成多个小步骤，与应用线程交替执行，减少单次停顿时间。

- 三色标记法：白色（未访问）、灰色（已访问，子对象未完全处理）、黑色（已完成）。
- 读写屏障 (Write Barrier)：在应用修改引用时记录变化，防止漏标。

增量回收 (Incremental GC)

问题背景

传统 GC（如标记-清除）需要暂停应用线程 (Stop-The-World)，导致明显的卡顿。

增量回收的核心思想

将 GC 工作拆分成多个小步骤，与应用线程交替执行，减少单次停顿时间。

- 三色标记法：白色（未访问）、灰色（已访问，子对象未完全处理）、黑色（已完成）。
- 读写屏障 (**Write Barrier**)：在应用修改引用时记录变化，防止漏标。
- 权衡：总工作量增加，但响应性更好。

垃圾回收算法对比总结

算法	核心思想	主要优点	主要缺点
引用计数	维护引用数，归零即回收	实时回收，无全局停顿	循环引用，额外开销
标记-清除	标记可达对象，清除未标记	实现简单，不移动对象	内存碎片
复制回收	存活对象复制到新半区	无碎片，分配快	空间利用率 50%
分代回收	按年龄分区，不同策略	高效，符合对象生命周期规律	实现复杂
增量回收	GC 与应用交替执行	低延迟，响应性好	总开销增加

目录

- ① 运行时的存储组织的作用与任务
- ② 存储分配策略
- ③ 活动记录
- ④ 函数调用与参数传递

⑤ 垃圾回收

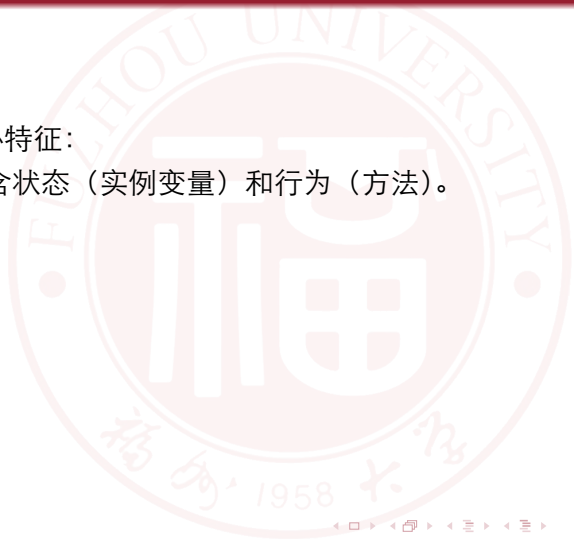
⑥ OOP 的运行时组织

⑦ 小结与作业

OOP 的运行时特征

面向对象程序在运行时具有以下核心特征：

- **对象 (Object)**：类的实例，包含状态（实例变量）和行为（方法）。



OOP 的运行时特征

面向对象程序在运行时具有以下核心特征：

- **对象 (Object)**：类的实例，包含状态（实例变量）和行为（方法）。
- **类 (Class)**：对象的模板，定义了对对象的属性和方法签名。

OOP 的运行时特征

面向对象程序在运行时具有以下核心特征：

- **对象 (Object)**：类的实例，包含状态（实例变量）和行为（方法）。
- **类 (Class)**：对象的模板，定义了对对象的属性和方法签名。
- **封装 (Encapsulation)**：数据与操作数据的方法绑定在一起。

OOP 的运行时特征

面向对象程序在运行时具有以下核心特征：

- **对象 (Object)**：类的实例，包含状态（实例变量）和行为（方法）。
- **类 (Class)**：对象的模板，定义了对对象的属性和方法签名。
- **封装 (Encapsulation)**：数据与操作数据的方法绑定在一起。
- **继承 (Inheritance)**：子类复用父类的属性和行为。

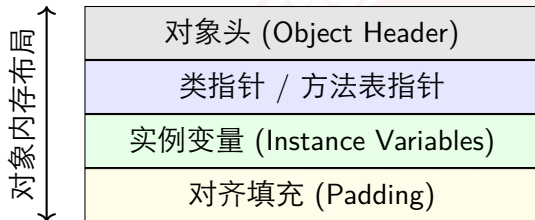
OOP 的运行时特征

面向对象程序在运行时具有以下核心特征：

- 对象 (**Object**)：类的实例，包含状态（实例变量）和行为（方法）。
- 类 (**Class**)：对象的模板，定义了对象的属性和方法签名。
- 封装 (**Encapsulation**)：数据与操作数据的方法绑定在一起。
- 继承 (**Inheritance**)：子类复用父类的属性和行为。
- 多态 (**Polymorphism**)：同一消息发送给不同对象可产生不同行为。

对象的存储组织

一个对象在内存中的典型布局：



- 对象头：哈希码、GC 年龄、锁状态等元数据。
- 类指针：指向该对象所属的类结构（或方法表）。
- 实例变量：继承自父类和自身定义的所有非静态字段。

方法的静态绑定 vs 动态绑定

静态绑定 (Static Binding)

- 编译时确定调用的方法。
- 适用于 `private`、`static`、`final` 方法。
- 效率高，无需运行时查找。

动态绑定 (Dynamic Binding)

- 运行时根据对象的实际类型确定方法。
- 实现多态的核心机制。
- 需要额外的运行时支持（如虚函数表）。

虚函数表 (Virtual Method Table, Vtable)

vtable 的作用

为每个类维护一张**方法指针表**，存储该类所有虚方法的入口地址。

Class A vtable

ptr to foo()

ptr to bar()

Class B vtable

ptr to foo() [覆盖]

ptr to bar() [继承]

- 每个对象含有一个指向 vtable 的指针。
- 子类继承父类 vtable，覆盖的方法替换指针。
- 调用虚方法时：
 - ① 通过对象找到 vtable
 - ② 通过偏移找到方法指针
 - ③ 跳转执行

动态绑定的执行过程

以 `obj->method()` 为例：

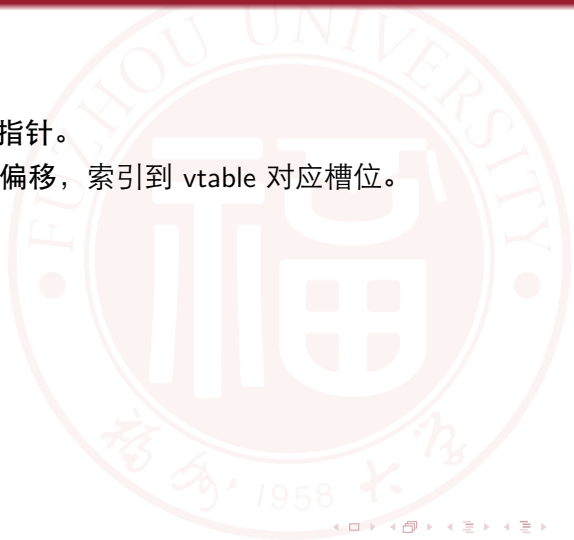
- ① 从对象指针 `obj` 中取出 **`vtable`** 指针。



动态绑定的执行过程

以 `obj->method()` 为例：

- ① 从对象指针 `obj` 中取出 **`vtable`** 指针。
- ② 根据 `method` 在类定义中的固定偏移，索引到 `vtable` 对应槽位。



动态绑定的执行过程

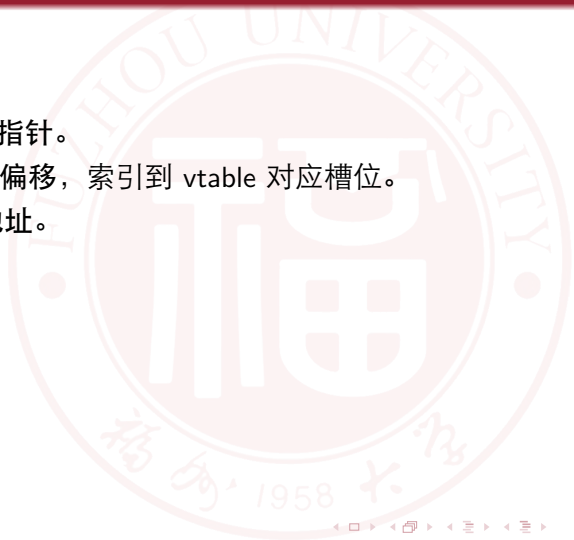
以 `obj->method()` 为例：

- ① 从对象指针 `obj` 中取出 **vtable** 指针。
- ② 根据 `method` 在类定义中的固定偏移，索引到 **vtable** 对应槽位。
- ③ 从槽位中取出实际的函数入口地址。

动态绑定的执行过程

以 `obj->method()` 为例：

- ① 从对象指针 `obj` 中取出 **`vtable`** 指针。
- ② 根据 `method` 在类定义中的固定偏移，索引到 `vtable` 对应槽位。
- ③ 从槽位中取出实际的函数入口地址。
- ④ 跳转执行。



动态绑定的执行过程

以 `obj->method()` 为例：

- ① 从对象指针 `obj` 中取出 **`vtable`** 指针。
- ② 根据 `method` 在类定义中的固定偏移，索引到 `vtable` 对应槽位。
- ③ 从槽位中取出实际的函数入口地址。
- ④ 跳转执行。

开销分析

相比直接调用，动态绑定增加了两次内存访问（取 `vtable` 指针 + 取函数指针）和一次间接跳转，但现代 CPU 的流水线优化使得这一开销通常可以忽略。

OOP 运行时组织总结

机制	说明
对象布局	对象头 + 类指针 + 实例变量 (+ 对齐填充)
类结构	存储静态字段、方法签名、父类引用、vtable
静态绑定	编译时确定，直接调用，效率高
动态绑定	运行时通过 vtable 查找，支持多态
单继承	vtable 线性排列，直接兼容
多继承	vtable 结构复杂，可能需要调整指针 (this-adjustment)

目录

- ① 运行时的存储组织的作用与任务
- ② 存储分配策略
- ③ 活动记录
- ④ 函数调用与参数传递

⑤ 垃圾回收

⑥ OOP 的运行时组织

⑦ 小结与作业

本讲小结（上）

- 运行时存储管理的核心使命：为程序中的名字建立与运行时地址的映射，管理数据对象的生命周期。
- 三大存储分配策略：
 - 静态分配：编译期确定，适用于全局/静态变量。
 - 栈式分配：LIFO，支持递归，通过活动记录管理局部数据。
 - 堆式分配：动态申请，需配合垃圾回收或手动管理。
- 活动记录与栈帧管理：
 - 关键寄存器 fp （帧基址）与 sp （栈顶）。
 - 序言建立栈帧，结语撤销栈帧。
 - 非局部量访问：静态链/Display 表（静态作用域）vs 控制链（动态作用域）。

本讲小结（下）

- 函数调用约定：x86（cdecl、stdcall、fastcall）与 RISC（ARM、MIPS、RISC-V）在参数传递、栈平衡、返回地址保存上的差异。
- 垃圾回收：引用计数、标记-清除、复制、分代、增量等算法各有适用场景与权衡。
- **OOP** 运行时：对象布局（对象头+类指针+实例变量），vtable 实现动态绑定与多态。

作业 1: 活动记录与调用过程分析 (上)

考虑如下 C 语言程序片段:

```
1  int add(int a, int b) {  
2      int sum = a + b;  
3      return sum;  
4  }  
5  int main() {  
6      int x = 3, y = 5;  
7      int r = add(x, y);  
8      return 0;  
9  }
```

任务 1:

- ① 画出从 main 调用 add 到 add 返回前, 栈中活动记录的完整布局 (标明参数、返回地址、旧 *fp*、局部变量等)。

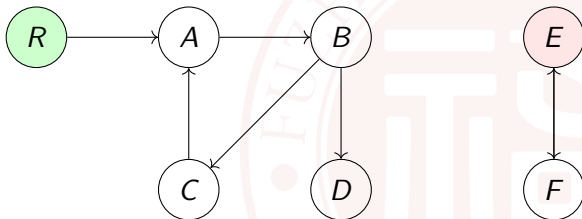
作业 1：活动记录与调用过程分析（下）

任务 2-3：

- ② 分别说明在 x86-64 (System V ABI) 和 RISC-V (RV64I) 调用约定下，参数 x、y 是如何传递给 add 的（寄存器还是栈？具体名称？）。
- ③ 若 add 是一个叶函数，在 RISC-V 中是否可以优化掉栈帧分配？请说明理由。

作业 2：垃圾回收算法分析（上）

假设某程序运行时刻的堆中对象引用关系如下图所示（根集合为 R ）：



任务 1：

- ① 若采用 引用计数法，请给出每个对象的引用计数初始值，并指出哪些对象存在循环引用问题、无法被回收。

作业 2：垃圾回收算法分析（下）

任务 2-3：

- ② 若采用 标记-清除法，请列出从根集合 R 出发的可达对象集合，以及被视为垃圾的对象集合。
- ③ 简述为什么现代 JVM 等运行时系统更倾向于采用分代回收策略，而非单一的标记-清除或引用计数。